

Does AI Code Detection Reduce to *Skill* Detection?

A preliminary, two-sided answer

Autonomous experiment (Claude Code) for Matthew

May 20, 2026

Abstract

We ask whether AI-code detectors track author **species** (human vs. LLM) or author **skill/style**. Across 2655 Codeforces Python samples (human solutions stratified by problem difficulty; LLM solutions from 3 models under natural/novice/expert prompts) and three detectors spanning the trained (DroidDetect) and statistical (Fast-DetectGPT, Binoculars) families, the answer is two-sided. **(1)** On the competitive-programming difficulty axis the confound is *absent*: the skill-matched vs. mismatched AUROC gap is $\Delta_{\text{confound}} \approx 0.003$, and within-species score slopes run *against* the naive skill prediction (mean human slope -0.17 SD). This vindicates the concern that terse competitive-programming style is the stylistic *opposite* of the idiomatic “senior-engineer” register that RLHF code models imitate. **(2)** The dominant effect, however, is not skill but **cross-source generalization failure of the trained detector**. As a control, our DroidDetect load reproduces the authors’ performance on their *own* DroidCollection Python test data (AUROC 0.999, human false-positive rate 0.017). Yet on clean human Codeforces Python from a *different* source (MatrixStudio) it labels **0.673** of ordinary human code AI at its native argmax operating point, and 0.830 of contamination-free pre-LLM elite code (Torvalds’ `git`, antirez’s Redis, Hipp’s SQLite, CPython)—with the most idiomatic code (CPython, 0.975) highest. So a near-perfect in-distribution detector becomes unusable on out-of-distribution human code, with a secondary idiomatic-style gradient on top. The zero-shot statistical detectors, having no training distribution, do not show this collapse. Deployment implication: the operative risk is brittle cross-distribution false positives, not competitive skill.

1 Question and hypotheses

Post-RLHF code models are tuned toward idiomatic, well-structured “senior-engineer” style. If detectors learned that style as the AI signal, then *AI-vs-human* detection partly reduces to *expert-vs-novice* detection. We pre-registered:

- **H_{species}**: detectors classify species; within-species skill has no effect (slopes ≈ 0 ; AUROC constant across skill cells).
- **H_{skill}**: detectors classify skill; among humans, higher skill $\rightarrow$ more “AI”; among LLMs, novice prompting $\rightarrow$ less “AI”.
- **H_{mixed}**: a weighted combination.

2 Data and methods

Human. Accepted Python-3 Codeforces solutions (MatrixStudio), difficulty bands easy (≤ 1200) and hard (≥ 1900). *Deviation from the original plan:* no public dataset co-locates human source

code with *author* Elo, so the human skill axis is problem difficulty—a noisy, and as it turns out stylistically *inverted*, proxy. **LLM.** 3 models (gemma-2-2b-it, Qwen2.5-3B-Instruct, and the frontier gpt-5.4-nano) under natural/novice/expert prompts on the same problems. **Detectors** (oriented higher = more AI): DroidDetect-Base-Binary (trained); Fast-DetectGPT and Binoculars (statistical; canonical vendored implementations). **Stats.** AUROC is primary (threshold-free); all CIs use a cluster bootstrap over *problems*; the within-species score slope (z-scored detector output regressed on the skill axis) is the mechanism test; Δ_{confound} is reported per detector.

3 Results

3.1 The confound is absent on the competitive-skill axis

Table 1 gives within-species slopes (SD units). Under H_{skill} both columns should be positive; instead the human slope is *negative* for DroidDetect (-0.30 , CI $[-0.53, -0.07]$) and Fast-DetectGPT (-0.30), and null for Binoculars (0.07). Higher competitive-programming difficulty skill is scored, if anything, *less* AI-like.

Detector	Family	Human slope (hard–easy)	LLM slope (novice→expert)
DroidDetect	trained	-0.30	-0.14
Fast-DetectGPT	statistical	-0.30	-0.54
Binoculars	statistical	0.07	-0.19

Table 1: Within-species skill slopes (z-scored detector score). Positive = skill-tracking.

The skill-matched vs. mismatched AUROC gap is essentially zero (Table 2): $\Delta_{\text{confound}} = 0.011$ (CI $[0.001, 0.021]$) for DroidDetect, and -0.004 and 0.002 for the statistical detectors—i.e. matching humans and LLMs on competitive skill does *not* collapse detector performance. On this axis the data favour H_{species} .

Detector	Family	AUROC matched	AUROC mismatched	Δ_{confound}
DroidDetect	trained	0.746	0.757	0.011
Fast-DetectGPT	statistical	0.704	0.700	-0.004
Binoculars	statistical	0.665	0.667	0.002

Table 2: AUROC by skill cell and Δ_{confound} (cluster-bootstrap over problems).

3.2 Cross-source generalization failure (the dominant effect)

Control — the detector is faithfully loaded. On the authors’ own DroidCollection Python test data our load achieves AUROC 0.999 with a 0.017 human false-positive rate and 0.983 machine recall—reproducing the published performance. So what follows is a property of the model, not a loading bug.

DroidDetect is a 2-class softmax classifier; its actual prediction is argmax (equivalently $P(\text{machine}) > 0.5$), with no tunable threshold. We report it at that native operating point and measure its *true* false-positive rate on ordinary human code rather than forcing one. On clean human Codeforces Python from a *different* source (MatrixStudio) the FPR explodes from 0.017 (in-distribution) to 0.673: that fraction of *ordinary* human solutions are called AI (vs. 0.843 of actual Codeforces

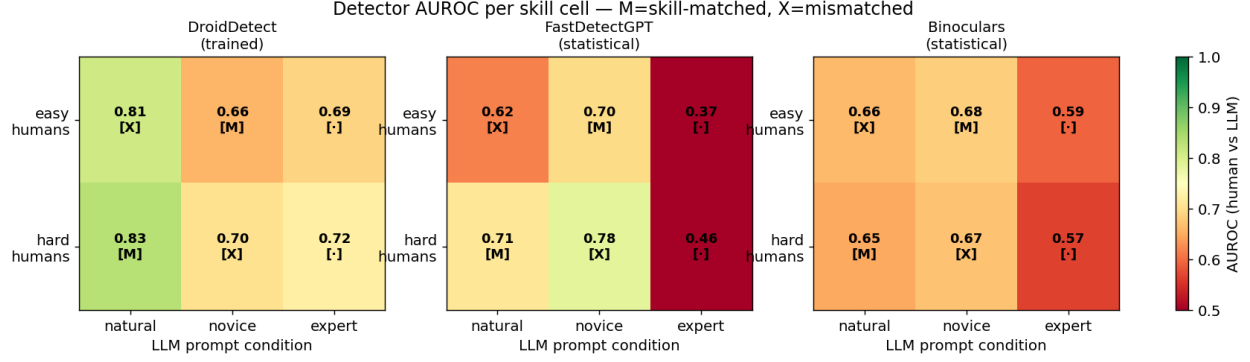


Figure 1: AUROC per (detector, difficulty band, prompt condition). M = skill-matched, X = mismatched.

LLM code—so the classifier barely separates the two on this out-of-distribution domain). On the provably-human pre-LLM elite code the rate is 0.830, and it rises monotonically with idiomatic polish: CPython library code 0.975, Hipp’s SQLite 0.825, antirez’s Redis 0.775, Torvalds’ original git 0.600. So there are two effects: a large *domain-shift* over-flagging of all human code here, and an *idiomatic-style* gradient on top (most-idiomatic CPython highest). The statistical detectors, which have no native threshold, are shown separately at a fixed 5%-FPR operating point for comparability; they flag the same code far less (Fast-DetectGPT 0.126, Binoculars 0.200).

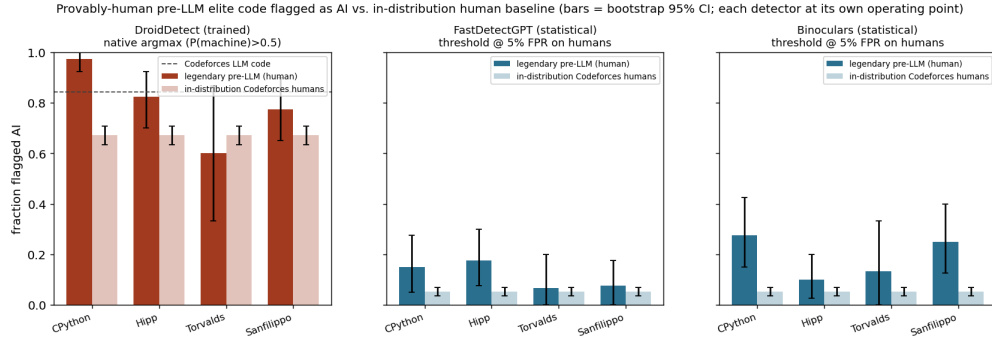


Figure 2: Provably-human pre-LLM elite code flagged as AI (solid), with the in-distribution ordinary-human baseline beside each (semitransparent); bootstrap 95% CIs. DroidDetect is shown at its native argmax operating point (baseline = its true human FPR, 0.673; dashed line = its rate on actual LLM code); the statistical detectors at a fixed 5%-FPR point.

3.3 Mechanism and confirmatory checks

Surface style features explain $R^2 = 0.108$ of DroidDetect’s score and 0.276 of Fast-DetectGPT’s—style matters but does not fully account for the trained detector. On the harder Codeforces problems human code is in fact *more* idiomatic (type-hint rate $0.594 \rightarrow 0.755$), which makes the negative human slope notable: more idiomatic *competitive* code is still not read as more AI.

A confirmatory cohort using real *author* Elo (AtCoder; low band $n = 45$ mean Elo 323, high band $n = 9$ mean Elo 2225) points the same way: the high–low score gap is negative for all detectors (e.g. DroidDetect -0.42 SD), though the thin high band leaves wide CIs.

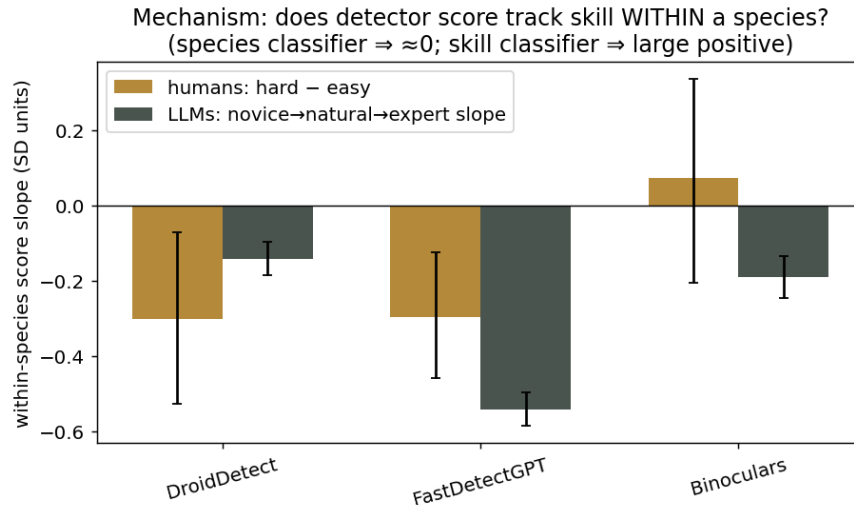


Figure 3: Within-species slopes: detector score vs. skill, per species.

4 Limitations

- **Problem difficulty / competitive rating $\neq$ idiomatic-code skill.** This is now the central finding, not just a caveat: the proxy is stylistically inverted.
- **Accepted-only human code** is a collider (rating $\rightarrow$ accept $\leftarrow$ difficulty); LLM code is unfiltered— asymmetric.
- **Small LLM panel** (gemma-2-2b, Qwen2.5-3B, gpt-5.4-nano); preliminary n .
- **MatrixStudio is a different human source** than DroidCollection’s training data; we cannot fully separate “source shift” from “idiomatic style” as causes of the over-flagging (the pre-LLM legendary probe is the contamination-free anchor, and the in-distribution control rules out a loading bug).
- **Thin AtCoder high-skill band** ($n = 9$).

5 Conclusion

Two findings, in order of importance. First and dominant: the trained detector is **not safe outside its training distribution**. Faithfully loaded (it scores AUROC 0.999 with a 0.017 FPR on the authors’ own data), it nonetheless labels 0.673 of ordinary human Codeforces code from a different source—and 0.830 of provably-human pre-LLM elite code (CPython 0.975)—as machine-generated. The zero-shot statistical detectors do not collapse this way. Second and secondary: “does detection reduce to *skill* detection?” — not to competitive-programming skill (the confound is absent and mildly inverts, because terse competitive style is the opposite of idiomatic style), but a weak idiomatic-style gradient rides on top of the domain-shift over-flagging (most-polished CPython flagged most). The deployment implication is that the first-order danger is brittle cross-distribution false positives—an academic-integrity tool calibrated on one corpus can flag the majority of legitimate human code from another. The clean follow-ups: evaluate trained detectors across held-out

human *sources*, and study the idiomatic-style axis with an author-reputation cohort on production code (e.g. GitHub by maintainer reputation).